

Session 0: Cost Safety & Account Preparation (Mandatory Step)

Create Budget Alert (Root Account)

Path:

Billing and Cost Management → Budgets → Create Budget

Steps:

1. Log in using the **root account**.
2. Navigate to **Billing and Cost Management**.
3. Go to **Budgets → Create Budget**.
4. Select **Cost Budget**.
5. Enter:
 - a. **Budget Name:** Zero-Spend-Alert
 - b. **Budget Amount:** \$0.01
 - c. **Threshold:** 100% of budget
 - d. **Email Alert:** Your personal email ID

AWS Budget Configuration

- Created a **Zero Spend Budget** to immediately notify on any AWS usage.
- Threshold set at **\$0.01** to avoid accidental billing.
- Email alerts configured.

Reason

This ensures **strict cost control** during infrastructure learning and testing and prevents unexpected AWS charges.

Key Architecture Decisions.

“How do you control AWS costs in non-production environments?”

Answer:

By configuring **AWS Budgets with alert thresholds**, especially a zero-spend alert, to immediately notify on any usage.

Session 1: IAM Setup for Project Access

Create IAM User for the Project

Path:

IAM → Users → Create User

Steps:

1. Create a new IAM user (example: project-admin).
2. Enable **Programmatic access**.
3. Attach policy:
 - a. AdministratorAccess (**temporary – only for learning**).
4. Create **Access Key and Secret Key**.
5. Configure AWS CLI using:

```
aws configure
```

⚠ Important Security Note:

- Access Key & Secret Key **must never be committed to GitHub**.
- Always remove them from documents before sharing.

One-Line Definition

IAM (Identity and Access Management):

Controls **who can access AWS resources and what actions they can perform**.

Key Architecture Decisions.**Q**

“Why should root account not be used for daily operations?”

Answer:

Root account has unrestricted access and should be protected; IAM users provide controlled, auditable access.

Session 2: Start Main Project – Network Foundation

Create VPC

What is a VPC?

A **Virtual Private Cloud (VPC)** is a logically isolated network in AWS where you define IP ranges, subnets, routing, and security.

Steps (Console):

1. Go to **VPC → Create VPC**.
2. Choose:
 - a. **VPC Only**
 - b. **CIDR Block:** 10.0.0.0/16
3. Enable:
 - a. DNS Hostnames
 - b. DNS Resolution
4. Name: proj-3tier-vpc

Why / Reason

This VPC acts as the **foundation network layer** for all application components.

Key Architecture Decisions.

Q:

“Why do we use /16 CIDR for VPC?”

Answer:

It provides enough IP range to create multiple subnets while allowing future scaling.

VPC dashboard <

AWS Global View ^L

Filter by VPC ▾

▼ **Virtual private cloud**

- Your VPCs
- Subnets
- Route tables
- Internet gateways
- Egress-only internet gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peering connections
- Route servers

▼ **Security**

- Network ACLs
- Security groups

Introducing VPC encryption control

Manage and enforce encryption settings across your Virtual Private Cloud (VPC) resources. This centralized control helps ensure compliance with security policies and data protection regulations.

[Create encryption control](#) ✕

Your VPCs

VPCs | VPC encryption controls - new

Your VPCs (1) [Info](#)

Find VPCs by attribute or tag

☐	Name	VPC ID	State	Encryption c...	Encryption control ...	Block Public...
☐	-	vpc-0ccefab8557f7a5ff	Available	-	-	Off

Select a VPC above

⋮ VPC > Your VPCs > Create VPC

☒ VPC only ☐ VPC and more

Name tag - optional

Creates a tag with a key of 'Name' and a value that you specify.

proj-3tier-vpc

IPv4 CIDR block [Info](#)

☒ IPv4 CIDR manual input ☐ IPAM-allocated IPv4 CIDR block

IPv4 CIDR

10.0.0.0/16

CIDR block size must be between /16 and /28.

IPv6 CIDR block [Info](#)

☒ No IPv6 CIDR block ☐ IPAM-allocated IPv6 CIDR block ☐ Amazon-provided IPv6 CIDR block ☐ IPv6 CIDR owned by me

Tenancy [Info](#)

Default ▾

VPC encryption control (\$) - new [Info](#)

Monitor mode provides visibility into encryption status without blocking traffic. Enforce mode prevents unencrypted traffic. [Additional charges apply](#) ^L

☒ None ☐ Monitor mode ☐ Enforce mode

See which resources in your VPC are unencrypted but allow the creation of

Requires all resources, except exclusions, in your VPC to be encryption-capable and

Enable VPC DNS Settings

Action:

- Enable **DNS Resolution**
- Enable **DNS Hostnames**
- Click **Save changes**

Why this is required:

- Required for assigning **DNS-based hostnames to EC2 instances**

- Mandatory for **Application Load Balancer (ALB)** to function correctly
- Required for **RDS endpoints** to resolve properly

One-line definition:

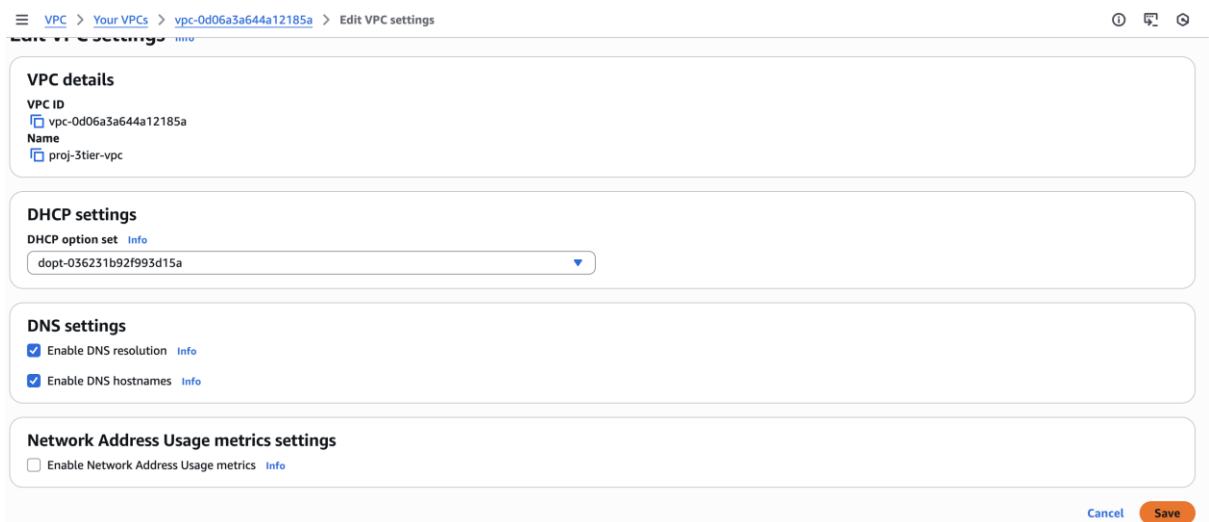
DNS Resolution and DNS Hostnames allow AWS resources inside a VPC to communicate using domain names instead of IP addresses.

Real-world scenario:

- If DNS Hostnames are disabled, ALB will not be able to route traffic using its DNS name.
- RDS endpoints (like db-name.cluster-xyz.amazonaws.com) will fail to resolve.

Key Architecture Decisions:

- **Question:** Why must DNS Hostnames be enabled in a VPC?
- **Answer:** To allow AWS-managed services like ALB and RDS to expose and resolve DNS endpoints.



The screenshot shows the 'Edit VPC settings' page in the AWS Management Console. The breadcrumb trail at the top indicates the path: VPC > Your VPCs > vpc-0d06a3a644a12185a > Edit VPC settings. The page is divided into four main sections:

- VPC details:** Shows the VPC ID as vpc-0d06a3a644a12185a and the Name as proj-3tier-vpc.
- DHCP settings:** Shows the DHCP option set as dopt-036231b92f993d15a.
- DNS settings:** Contains two checkboxes, both of which are checked: 'Enable DNS resolution' and 'Enable DNS hostnames'. Each checkbox has an 'Info' link next to it.
- Network Address Usage metrics settings:** Contains one unchecked checkbox: 'Enable Network Address Usage metrics', with an 'Info' link next to it.

At the bottom right of the page, there are 'Cancel' and 'Save' buttons.

SESSION 3 – INTERNET CONNECTIVITY (IGW + Terraform Import)

VPC > Internet gateways > igw-01952ba2b33377693

Internet gateway igw-01952ba2b33377693 successfully attached to vpc-0d06a3a644a12185a

VPC dashboard

AWS Global View

Filter by VPC

Virtual private cloud

Your VPCs

Subnets

Route tables

Internet gateways

Egress-only internet gateways

DHCP option sets

Elastic IPs

Managed prefix lists

NAT gateways

Peering connections

Route servers

Security

igw-01952ba2b33377693 / proj-3tier-igw

Actions

Details

Internet gateway ID

igw-01952ba2b33377693

State

Attached

VPC ID

vpc-0d06a3a644a12185a | proj-3tier-vpc

Owner

211811255273

Tags (1)

Search tags

Key	Value
Name	proj-3tier-igw

Manage tags

Internet gateway igw-01952ba2b33377693 successfully attached to vpc-0d06a3a644a12185a

igw-01952ba2b33377693 / proj-3tier-igw

Actions

Details

Internet gateway ID

igw-01952ba2b33377693

State

Attached

VPC ID

vpc-0d06a3a644a12185a | proj-3tier-vpc

Owner

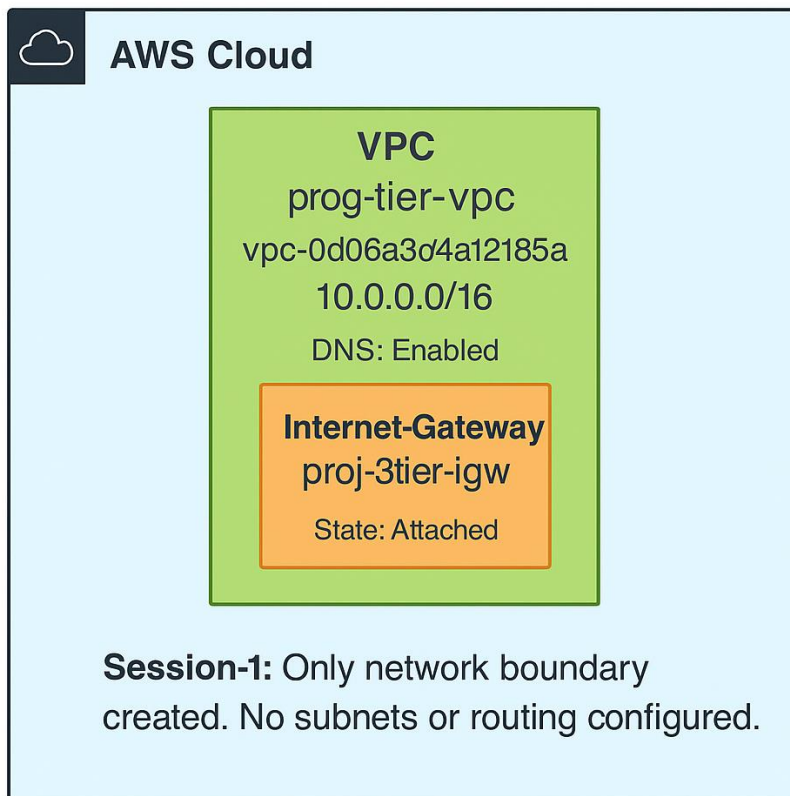
211811255273

Tags (1)

Search tags

Key	Value
Name	proj-3tier-igw

Manage tags



INTERNET GATEWAY (IGW)

Labels to Add (MANDATORY)

VPC Details

- **Name:** proj-3tier-vpc
- **VPC ID:** vpc-0d06a3a644a12185a
- **CIDR:** 10.0.0.0/16
- **DNS:** Enabled

Internet Gateway Details

- **Name:** proj-3tier-igw
- **State:** Attached

◆ SESSION 4 – TERRAFORM PROJECT STRUCTURE

Terraform Setup (Import Existing Resources)

Create Terraform Directory Structure

```
mkdir terraform
cd terraform
```

```
mkdir -p modules/vpc
tree
```

Output:

```
.
├── modules
│   └── vpc
```

3 directories, 0 files

Terraform Backend Configuration

```
cat <<EOF > backend.tf
terraform{
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
EOF
```


Terraform Provider Configuration

```
cat <<EOF > provider.tf
provider "aws" {
  region = var.aws_region
}
EOF
```

Root Variables

```
cat <<EOF > variables.tf
variable "aws_region" {
  description = "AWS region"
  type       = string
}

variable "environment" {
  description = "Environment name"
  type       = string
}
EOF
```

Terraform Variables File

```
cat <<EOF > terraform.tfvars
aws_region = "ap-south-1"
environment = "dev"
EOF
```

Root Module Configuration

```
cat <<EOF > main.tf
module "vpc" {
  source = "../modules/vpc"

  vpc_name = "proj-3tier-vpc"
  cidr     = "10.0.0.0/16"

  tags = {
    Project   = "AWS-3Tier"
    Environment = var.environment
  }
}
EOF
```

Root Outputs

```
cat <<EOF > outputs.tf
output "vpc_id" {
  value = module.vpc.vpc_id
}

output "igw_id" {
  value = module.vpc.igw_id
}
EOF
```

VPC Module – Variables

```
cd modules/vpc

cat <<EOF > variables.tf
variable "vpc_name" {
```

```
    type = string
  }

  variable "cidr" {
    type = string
  }

  variable "tags" {
    type = map(string)
  }
EOF
```

VPC Module – Main Configuration

```
cat <<EOF > main.tf
resource "aws_vpc" "this" {
  cidr_block      = var.cidr
  enable_dns_support = true
  enable_dns_hostnames = true

  tags = merge(
    var.tags,
    {
      Name = var.vpc_name
    }
  )
}

resource "aws_internet_gateway" "this" {
  vpc_id = aws_vpc.this.id

  tags = merge(
    var.tags,
    {
      Name = "proj-3tier-igw"
    }
  )
}
```

```
}  
EOF
```

VPC Module – Outputs

```
cat <<EOF > outputs.tf  
output "vpc_id" {  
    value = aws_vpc.this.id  
}  
  
output "igw_id" {  
    value = aws_internet_gateway.this.id  
}  
EOF
```

Verify Terraform Structure

```
cd ../../..  
tree
```

Output:

```
.  
├── backend.tf  
├── main.tf  
├── modules  
│   ├── vpc  
│   │   ├── main.tf  
│   │   ├── outputs.tf  
│   │   └── variables.tf  
├── outputs.tf  
├── provider.tf  
└── terraform.tfvars
```

└─ variables.tf

Terraform Initialization

```
terraform init
```

Output preserved exactly as executed.

Terraform Workspace

```
terraform workspace new dev
terraform workspace select dev
```

Import Existing VPC (Initial Credential Issue)

```
terraform import module.vpc.aws_vpc.this vpc-0d06a3a644a12185a
```

Error (as encountered):

Error: No valid credential sources found

AWS CLI Configuration (Credentials Fixed)

```
aws configure
```

 Access Key & Secret masked intentionally

Verify AWS Identity

```
aws sts get-caller-identity
```

Output:

```
{  
  "UserId": "",  
  "Account": "",  
  "Arn": ""  
}
```

Import VPC Successfully

```
terraform import module.vpc.aws_vpc.this vpc-0d06a3a644a12185a
```

 **Import successful**

Import Internet Gateway

```
terraform import module.vpc.aws_internet_gateway.this igw-01952ba2b33377693
```

 **Import successful**

Terraform Plan (Tag Drift Only)

```
terraform plan
```

Result:

Plan: 0 to add, 2 to change, 0 to destroy.

Final Sync Check

terraform plan

Result:

No changes. Your infrastructure matches the configuration.

Next Step (Not Executed Yet)

Now, going forward to create:

Subnet + Route Table

AWS Console → VPC → Subnets → Create Subnet

Key Takeaway (One-Line Definition)

Terraform Import:

Used to bring existing AWS resources under Terraform state without recreating them.

- ◆ SESSION 5 – SUBNET DESIGN (PUBLIC & PRIVATE)

Subnet:

Subnet 1 of 1

Subnet name

Create a tag with a key of 'Name' and a value that you specify.

proj-public-subnet-1

The name can be up to 256 characters long.

Availability Zone [Info](#)

Choose the zone in which your subnet will reside, or let Amazon choose one for you.

Europe (Stockholm) / eun1-az1 (eu-north-1a) ▼

IPv4 VPC CIDR block [Info](#)

Choose the VPC's IPv4 CIDR block for the subnet. The subnet's IPv4 CIDR must lie within this block.

10.0.0.0/16 ▼

IPv4 subnet CIDR block

10.0.1.0/24

256 IPs

< > ^ v

▼ **Tags - optional**

Key

Q Name X

Value - optional

Q proj-public-subnet-1 X

Remove

Add new tag

You can add 49 more tags.

Objective:

Create public and private subnets across multiple Availability Zones for high availability.

☑ STEP 3: Create PUBLIC SUBNET – 2

Path:

AWS Console → VPC → Subnets → Create subnet

Configuration

Field	Value
VPC	proj-3tier-vpc
Subnet name	proj-public-subnet-2
Availability Zone	eu-north-1b
IPv4 CIDR	10.0.2.0/24

👉 Click **Create**

Why (One-Line):

Second public subnet in another AZ ensures **high availability for ALB**.

STEP 4: Create PRIVATE SUBNET – 1

Path:

AWS Console → VPC → Subnets → Create subnet

Configuration

Field	Value
VPC	proj-3tier-vpc
Subnet name	proj-private-subnet-1
Availability Zone	eu-north-1a
IPv4 CIDR	10.0.11.0/24

 Click **Create**

Why (One-Line):

Private subnet for backend EC2 / application tier (no direct internet access).

STEP 5: Create PRIVATE SUBNET – 2

Path:

AWS Console → VPC → Subnets → Create subnet

Configuration

Field	Value
VPC	proj-3tier-vpc
Subnet name	proj-private-subnet-2
Availability Zone	eu-north-1b
IPv4 CIDR	10.0.12.0/24

 Click **Create**

Why (One-Line):

Second private subnet for **fault tolerance** and future Auto Scaling Group usage.

POST-CREATION STEP (MANDATORY)

Enable Auto-Assign Public IP for Public Subnets

Path:

VPC → Subnets

→ Select Public Subnet

→ Actions → Edit subnet settings

☒ Enable:

- **Auto-assign IPv4 public IP**

 Save changes

Why:

Public EC2 instances (like Bastion / ALB) must receive a public IP automatically.

SESSION 5 SUMMARY

Subnet	Type	AZ	CIDR
proj-public-subnet-1	Public	eu-north-1a	10.0.1.0/24
proj-public-subnet-2	Public	eu-north-1b	10.0.2.0/24
proj-private-subnet-1	Private	eu-north-1a	10.0.11.0/24
proj-private-subnet-2	Private	eu-north-1b	10.0.12.0/24

VPC > Subnets > subnet-04a9f9d40c1e98614 > Edit subnet settings

Subnet

Subnet ID

subnet-04a9f9d40c1e98614

Name

proj-public-subnet-2

Auto-assign IP settings

Info

Enable AWS to automatically assign a public IPv4 or IPv6 address to a new primary network interface for an instance in this subnet.

☒ Enable auto-assign public IPv4 address

Info

☐ Enable auto-assign customer-owned IPv4 address

Info

Option disabled because no customer owned pools found.

Resource-based name (RBN) settings

Info

Specify the hostname type for EC2 instances in this subnet and optional RBN DNS query settings.

☐ Enable resource name DNS A record on launch

Info

☐ Enable resource name DNS AAAA record on launch

Info

Hostname type

Info

☐ Resource name

☒ IP name

rtb-086a14f262b40e8d8 / proj-public-rt

Actions

Details

Info

Route table ID

rtb-086a14f262b40e8d8

VPC

vpc-0d06a3a644a12185a | proj-3tier-vpc

Main

No

Owner ID

211811255273

Explicit subnet associations

-

Edge associations

-

Routes

Subnet associations

Edge associations

Route propagation

Tags

Routes (2)

Both

Edit routes

Filter routes

< 1 >

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	igw-01952ba2b33377693	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

SESSION 6 – ROUTING (PUBLIC ROUTE TABLE)

✗ Do NOT enable auto-assign public IPv4 for private subnets

Why (one line):

Private subnets must never be directly reachable from the internet.

STEP 1: Create PUBLIC ROUTE TABLE

Console Path

AWS Console → VPC → Route Tables → Create route table

Configuration

Field	Value
Name	proj-public-rt
VPC	proj-3tier-vpc

👉 Click **Create route table**

STEP 2: Add Internet Route (0.0.0.0/0)

Console Path

Route Tables → proj-public-rt → Routes → Edit routes → Add route

Destination	Target
0.0.0.0/0	Internet Gateway (proj-3tier-igw)

👉 Save changes

Why:

This allows **public subnets** to send and receive internet traffic.

STEP 3: Associate PUBLIC SUBNETS

Console Path

Route Tables → proj-public-rt → Subnet associations → Edit subnet associations

Select

- ☒ proj-public-subnet-1
- ☒ proj-public-subnet-2

👉 Save associations

Why:

Only subnets associated with this route table become **public**.

🔍 Validation Checklist (Very Important)

- Public subnets:
 - ✓ Have route to **IGW**
 - ✓ Auto-assign public IP **enabled**
- Private subnets:
 - ✗ NOT associated with public route table
 - ✗ No IGW route

🔄 Key Architecture Decisions

Scenario:

“EC2 in public subnet has public IP but no internet access.”

Root Cause:

Public subnet not associated with a route table having IGW route.

📄 One-line Definition (for revision)

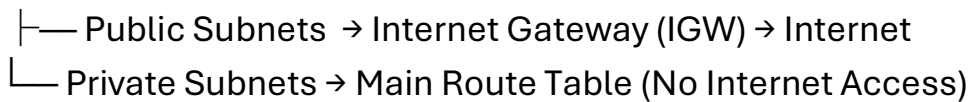
Public Route Table:

A route table that contains a default route (0.0.0.0/0) pointing to an Internet Gateway.

If you're ready, next you should send notes for:

Architecture Flow (Conceptual View)

VPC



Associate PUBLIC SUBNETS

What we are doing

We associate **public subnets** with the **public route table** that has an Internet Gateway route.

Public Subnets Flow

Public Subnet



Public Route Table



Internet Gateway (IGW)



Internet

Why (one line):

This allows EC2 instances in public subnets to access the internet and be accessed from the internet.

Private Subnets Flow

Private Subnet

↓

Main Route Table

↓

(No IGW route)

Why (one line):

Private subnets must not have direct internet access for security reasons.

Important Rule (Very Important for Interviews)

✗ Private subnets must NEVER be associated with a route table that has an IGW route

Key Architecture Decisions.

Question:

“Why can’t my private EC2 access the internet even though it is inside a VPC?”

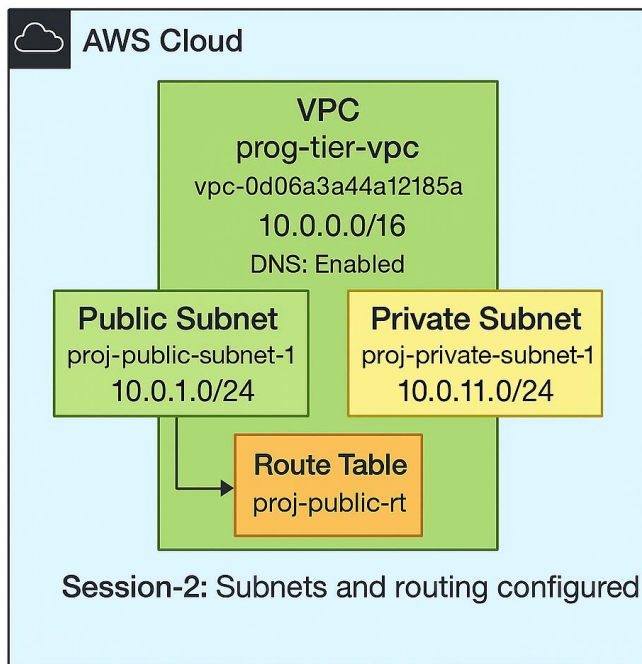
Answer:

Because private subnets are associated with the main route table that does not have an Internet Gateway route.

One-Line Definition (Revision)

Subnet Association:

Subnet association defines which route table controls traffic flow for a subnet.



Import Terraform Code:

Create NAT GW:

VPC > NAT gateways > nat-1c803e8cbf90c9ada

nat-1c803e8cbf90c9ada / proj-3tier-nat

PC dashboard

NS Global View

Filter by VPC

Virtual private cloud

- our VPCs
- Subnets
- Route tables
- Internet gateways
- Press-only internet gateways
- EC2 option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peer connections
- Route servers

Security

- Network ACLs
- Security groups

Details

NAT gateway ID
 nat-1c803e8cbf90c9ada

NAT gateway ARN
 arn:aws:ec2:eu-north-1:211811255273:natgateway/nat-1c803e8cbf90c9ada

VPC
 vpc-0d06a3a644a12185a / proj-3tier-vpc

Availability mode
 Regional

Connectivity type
 Public

Method of EIP allocation
 Automatic

State
 Pending

Created
 Monday, December 15, 2025 at 15:40:04 GMT+5:30

State message
 -

Deleted
 -

IP addresses | Monitoring | Flow logs | Tags

Associated IP addresses

Search

1

No associated IP addresses found. If this NAT gateway was recently created with the automatic allocation method, your IP addresses are still being allocated. Click the refresh button to view them

Create Elastic Ip in EC2:

Status is pending in NAT GW:

Now create pvt route table and attached IP and associate subnet:

The screenshot displays the AWS VPC console interface. At the top, a green notification bar states: "Updated routes for rtb-01950229dbfa4a057 / proj-private-rt successfully". Below this, the main header shows the selected route table: "rtb-01950229dbfa4a057 / proj-private-rt".

The "Details" tab is active, showing the following information:

- Route table ID:** rtb-01950229dbfa4a057
- Main:** No
- Explicit subnet associations:** -
- Edge associations:** -
- VPC:** vpc-0d06a3a644a12185a | proj-3tier-vpc
- Owner ID:** 211811255273

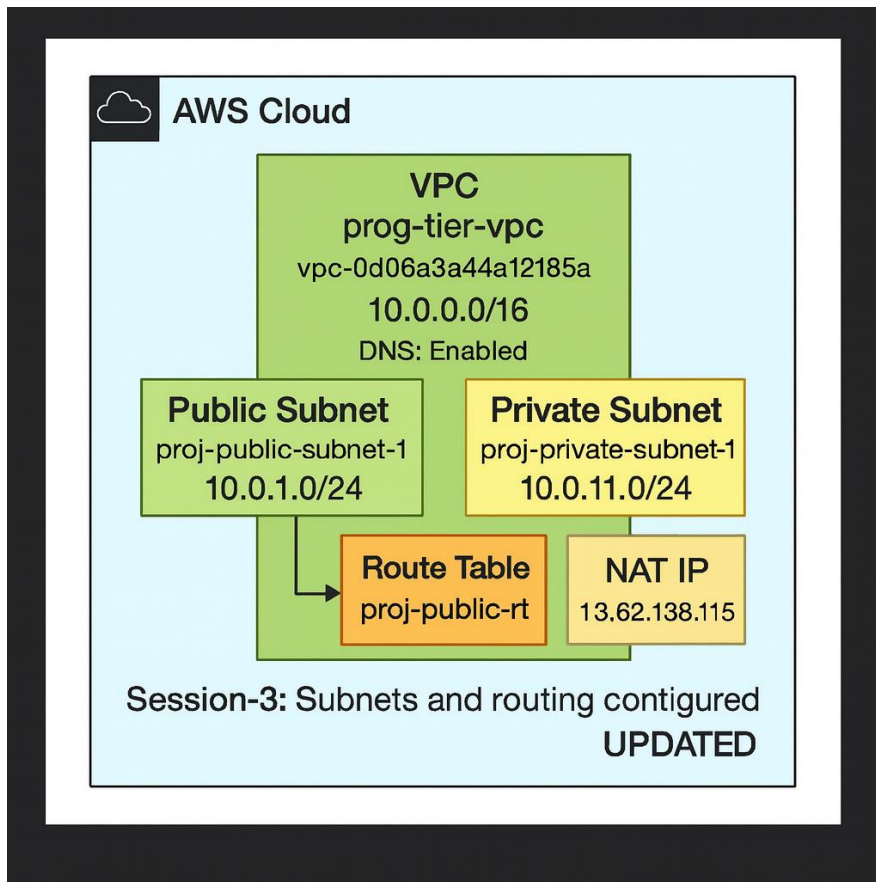
Below the details, the "Routes" tab is selected, showing a table of routes:

Destination	Target	Status	Propagated	Route Origin
0.0.0.0/0	nat-1c803e8cbf90c9ada	Active	No	Create Route
10.0.0.0/16	local	Active	No	Create Route Table

At the bottom, the "Route tables (5)" section provides a summary list:

Name	Route table ID	Explicit subnet associ...	Edge associations	Main	VPC
-	rtb-008a97179f3594b04	-	-	Yes	vpc-0ccefab8557
-	rtb-024e3dfa034a3d9b7	-	-	Yes	vpc-0d06a3a644
proj-private-rt	rtb-01950229dbfa4a057	2 subnets	-	No	vpc-0d06a3a644
-	rtb-035b8183cf1f65980	-	nat-1c803e8cbf90c...	No	vpc-0d06a3a644
proj-public-rt	rtb-086a14f262b40e8d8	2 subnets	-	No	vpc-0d06a3a644

- Public subnets → IGW
- Private subnets → NAT → IGW



Now update tf

code:

```
terraform import \  
module.vpc.aws_route.private_nat \  
rtb-01950229dbfa4a057_0.0.0.0/0
```

Instance Level Secuirty:

ALB SC --> EC2 SG --> DBSC

EC2 > Security Groups > Create security group

Basic details

Security group name [Info](#)

proj-db-sg

Name cannot be edited after creation.

Description [Info](#)

Database security group

VPC [Info](#)

vpc-0d06a3a644a12185a (proj-3tier-vpc)

Inbound rules [Info](#)

This security group has no inbound rules.

Add rule

Outbound rules [Info](#)

Type Info	Protocol Info	Port range Info	Destination Info	Description - optional Info
MySQL/Aurora	TCP	3306	Custom	sg-002b5f602f4d511 sg-002b5f602f4d511bc

In **Security Groups list**, verify:

✓ ALB SG

- Inbound: 80, 443 from 0.0.0.0/0

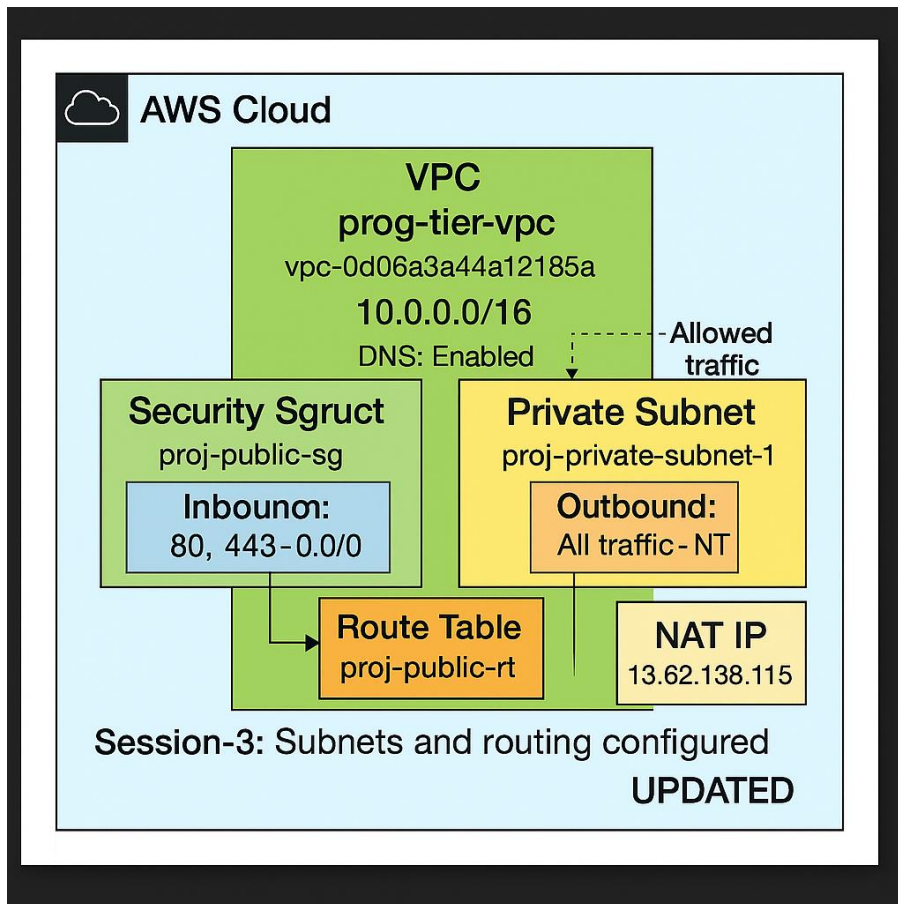
✓ APP SG

- Inbound: 80/8080 from **ALB SG**
- ✗ No public CIDR

✓ DB SG

- Inbound: 3306 from **APP SG**
- ✗ No public CIDR

=====



Creating Target Group and LB:

EC2 > Target groups > Create target group

Step 3
Review and create

Settings - immutable
Choose a target type and the load balancer and listener will route traffic to your target. These settings can't be modified after target group creation.

Target type
Indicate what resource type you want to target. Only the selected resource type can be registered to this target group.

☒ **Instances**
Supports load balancing to instances in a VPC. Integrate with Auto Scaling Groups or ECS services for automatic management.
Suitable for: ALB NLB GWLB

☐ **IP addresses**
Supports load balancing to VPC and on-premises resources. Facilitates routing to IP addresses and network interfaces on the same instance. Supports IPv6 targets.
Suitable for: ALB NLB GWLB

☐ **Lambda function**
Supports load balancing to a single Lambda function. ALB required as traffic source.
Suitable for: ALB

☐ **Application Load Balancer**
Allows use of static IP addresses and PrivateLink with an Application Load Balancer. NLB required as traffic source.
Suitable for: NLB

Target group name
Name must be unique per Region per AWS account.
proj-app-tg
Accepts: a-z, A-Z, 0-9, and hyphen (-). Can't begin or end with hyphen. 1-32 total characters; Count: 11/32

Protocol
Protocol for communication between the load balancer and targets.
HTTP

Port
Port number where targets receive traffic. Can be overridden for individual targets during registration.
80

Accepts: a-z, A-Z, 0-9, and hyphen (-). Can't begin or end with hyphen. 1-32 total characters; Count: 1 / 32

Protocol

Protocol for communication between the load balancer and targets.

HTTP

Port

Port number where targets receive traffic. Can be overridden for individual targets during registration.

80

1-65535

IP address type

Only targets with the indicated IP address type can be registered to this target group.

☒ IPv4

Each instance has a default network interface (eth0) that is assigned the primary private IPv4 address. The instance's primary private IPv4 address is the one that will be applied to the target.

☐ IPv6

Each instance you register must have an assigned primary IPv6 address. This is configured on the instance's default network interface (eth0). [Learn more](#)

VPC

Select the VPC with the instances that you want to include in the target group. Only VPCs that support the IP address type selected above are available in this list.

vpc-0d06a3a644a12185a (proj-3tier-vpc)
10.0.0.0/16



[Create VPC](#)

Protocol version

☒ HTTP1

Send requests to targets using HTTP/1.1. Supported when the request protocol is HTTP/1.1 or HTTP/2.

☐ HTTP2

Send requests to targets using HTTP/2. Supported when the request protocol is HTTP/2 or gRPC, but gRPC-specific features are not available.

☐ gRPC

Health check protocol

HTTP

Health check path

Use the default path of "/" to perform health checks on the root, or specify a custom path if preferred.

/

Up to 1024 characters allowed.

► Advanced health check settings

Target optimizer - optional [Info](#)

Use a target control port when the target has a strict concurrency limit.

Target control port

The port on which the target communicates its capacity. This value can't be modified after target group creation.

Enter target control port

Valid range: 1-65535

Attributes

Certain default attributes will be applied to your target group. You can view and edit them after creating the target group.

Create Application Load Balancer [Info](#)

The Application Load Balancer distributes incoming HTTP and HTTPS traffic across multiple targets such as Amazon EC2 instances, microservices, and containers, based on request attributes. When the load balancer receives a connection request, it evaluates the listener rules in priority order to determine which rule to apply, and if applicable, it selects a target from the target group for the rule action.

► How Application Load Balancers work

Basic configuration

Load balancer name

Name must be unique within your AWS account and can't be changed after the load balancer is created.

proj-alb

A maximum of 32 alphanumeric characters including hyphens are allowed, but the name must not begin or end with a hyphen.

Scheme [Info](#)

Scheme can't be changed after the load balancer is created.

☒ **Internet-facing**

- Serves Internet-facing traffic.
- Has public IP addresses.
- DNS name resolves to public IPs.
- Requires a public subnet.

☐ **Internal**

- Serves internal traffic.
- Has private IP addresses.
- DNS name resolves to private IPs.
- Compatible with the IPv4 and Dualstack IP address types.

Load balancer IP address type [Info](#)

Select the front-end IP address type to assign to the load balancer. The VPC and subnets mapped to this load balancer must include the selected IP address types. Public IPv4 addresses have an additional cost.

☒ **IPv4**

Includes only IPv4 addresses.

☐ **Dualstack**

on-premises targets, or if using VPC peering. To confirm the VPC for your targets, view [target groups](#).

vpc-0d06a3a644a12185a (proj-3tier-vpc)
10.0.0.0/16

Create VPC

IP pools [Info](#)

You can optionally choose to configure an IPAM pool as the preferred source for your load balancers IP addresses. Create or view Pools in the [Amazon VPC IP Address Manager console](#).

☐ **Use IPAM pool for public IPv4 addresses**
The IPAM pool you choose will be the preferred source of public IPv4 addresses. If the pool is depleted IPv4 addresses will be assigned by AWS.

Availability Zones and subnets [Info](#)

Select at least two Availability Zones and a subnet for each zone. A load balancer node will be placed in each selected zone and will automatically scale in response to traffic. The load balancer routes traffic to targets in the selected Availability Zones only.

☒ eu-north-1a (eu1-az1)

Subnet

Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently.

subnet-0e123e973c24fdecf
IPv4 subnet CIDR: 10.0.1.0/24

proj-public-subnet-1

☒ eu-north-1b (eu1-az2)

Subnet

Only CIDR blocks corresponding to the load balancer IP address type are used. At least 8 available IP addresses are required for your load balancer to scale efficiently.

subnet-04a9f9d40c1e98614
IPv4 subnet CIDR: 10.0.2.0/24

proj-public-subnet-2

Security groups [Info](#)

A security group is a set of firewall rules that control the traffic to your load balancer. Select an existing security group, or you can [create a new security group](#).

Select up to 5 security groups

proj-alb-sg
sg-0ce7e7640c387f725 VPC: vpc-0d06a3a644a12185a

Listeners and routing [Info](#)

A listener is a process that checks for connection requests using the port and protocol you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets.

► Listener HTTP:80 Remove

Add listener

You can add up to 49 more listeners.

► Load balancer tags - optional

Consider adding tags to your load balancer. Tags enable you to categorize your AWS resources so you can more easily manage them. The 'Key' is required, but 'Value' is optional. For example, you can have Key = production-webserver, or Key = webserver, and Value = production.

Optimize with service integrations - optional [Info](#)

Optimize your load balancing architecture by integrating AWS services with this load balancer at launch. You can also add these and other services after your load balancer is created by reviewing the load balancer's "Integrations" tab.

Verify:

EC2 > Load balancers > proj-alb

▼ Elastic Block Store
 Volumes
 Snapshots
 Lifecycle Manager

▼ Network & Security
 Security Groups
 Elastic IPs
 Placement Groups
 Key Pairs
 Network Interfaces

▼ Load Balancing
 Load Balancers
 Target Groups
 Trust Stores

▼ Auto Scaling
 Auto Scaling Groups

Settings

Scheme Internet-facing	Hosted zone Z23TAZ6LKFMNIO	Availability Zones subnet-04a9f9d40c1e98614 eu-north-1b (eu1-az2) subnet-0e123e973c24fdecf eu-north-1a (eu1-az1)	Date created December 15, 2025, 19:03 (UTC+05:30)
Load balancer ARN arn:aws:elasticloadbalancing:eu-north-1:211811255273:loadbalancer/app/proj-alb/7ad611ec77590724		DNS name Info proj-alb-1126198971.eu-north-1.elb.amazonaws.com (A Record)	

< **Listeners and rules** Network mapping Resource map Security Monitoring Integrations Attributes Capacity >

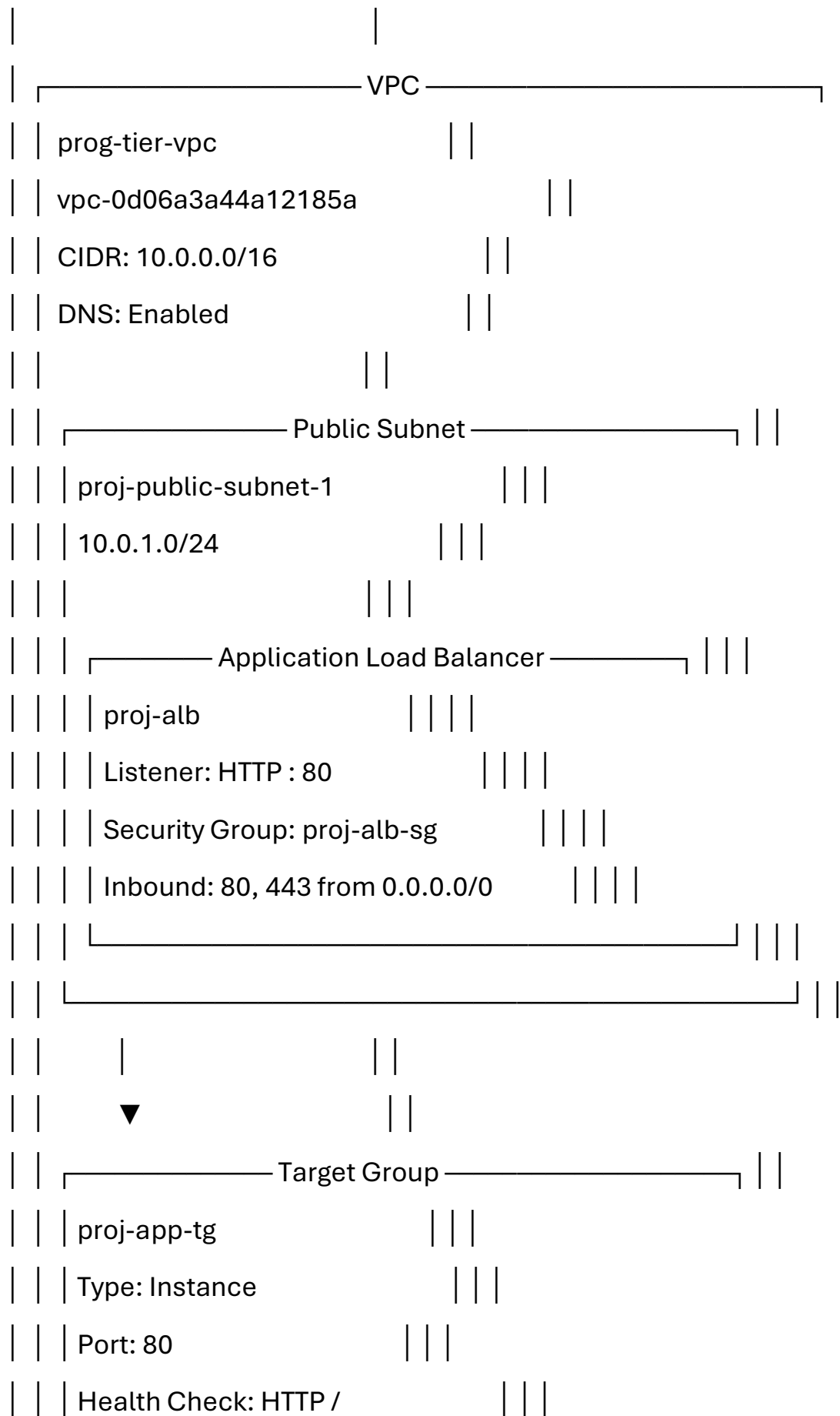
Listeners and rules (1) [Info](#) [Manage rules](#) [Manage listener](#) [Add listener](#)

A listener checks for connection requests on its configured protocol and port. Traffic received by the listener is routed according to the default action and any additional rules.

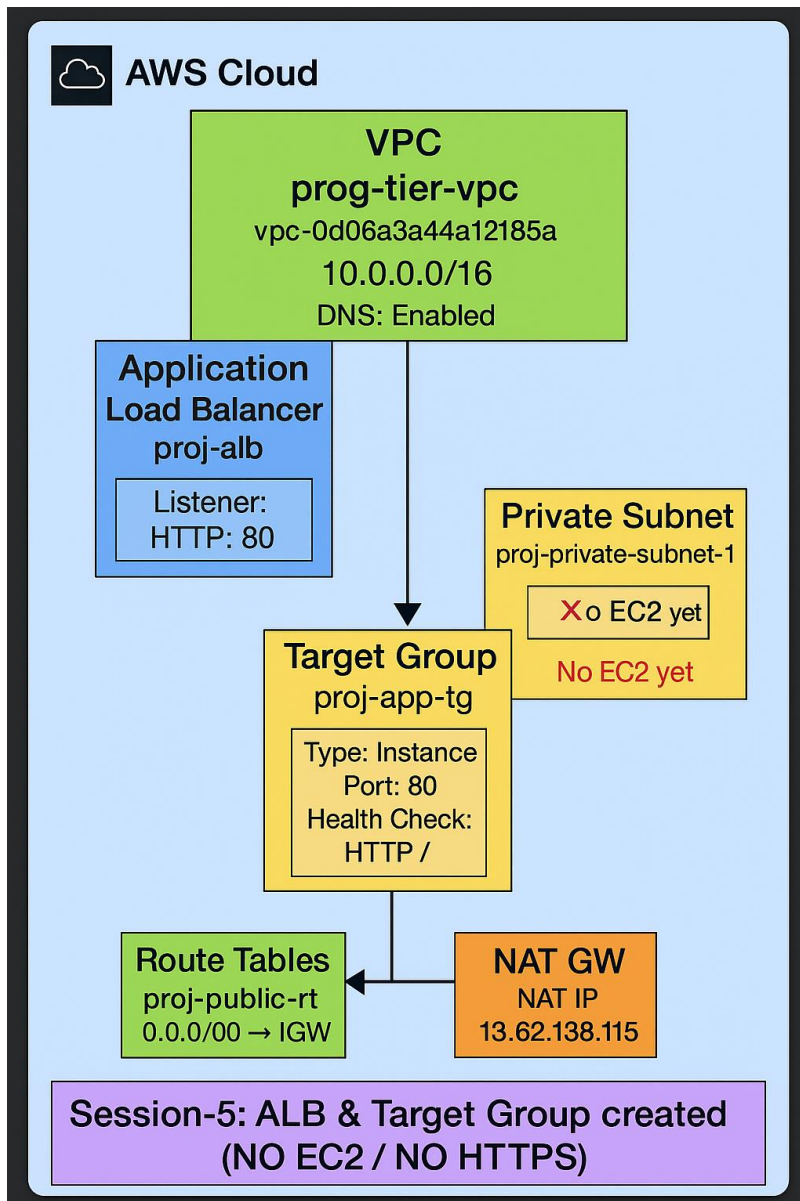
☐	Protocol:Port	Default action	Rules	ARN	Security policy	Default
☐	HTTP:80	• Forward to target group proj-app-tg L : 1 (100%) Target group stickiness: Off	1 rule	ARN	Not applicable	Not app

Verify Target Group:

We create the target group first, then attach it to the ALB listener.
 Instances are registered later via Auto Scaling Group.



Targets:	✗	NONE REGISTERED	
Private Subnet			
proj-private-subnet-1			
10.0.11.0/24			
Security Group:	proj-app-sg		
✗	No EC2 yet		
Route Tables			
proj-public-rt	NAT GW		
0.0.0.0/0 → IGW	NAT IP		
		13.62.138.115	
Session-5: ALB & Target Group created (NO EC2 / NO HTTPS)			



EC2 Instance Creation and serve the traffic:



AWS Cloud

VPC

prog-tier-vpc

vpc-0d06a304a12185a

10.0.0.0/16

DNS: Enabled

Internet-Gateway

proj-3tier-igw

State: Attached

Session-1: Only network boundary created. No subnets or routing configured.

LANCH TEMPLATE:

Name

Auto Scaling group name
Enter a name to identify the group.

proj-app-lt

Must be unique to this account in the current Region and no more than 255 characters.

Launch template [Info](#) [Switch to launch configuration](#)

Launch template
Choose a launch template that contains the instance-level settings, such as the Amazon Machine Image (AMI), instance type, key pair, and security groups.

proj-app-lt

[Create a launch template](#)

Version
Default (1)

[Create a launch template version](#)

Description from-existing-ec2	Launch template proj-app-lt lt-0b0d979efebb1941f	Instance type -
AMI ID ami-0b46816ffa1234887	Security groups -	Request Spot Instances No
Key pair name ThreeTierApp	Security group IDs -	

Integrate with other services - optional [Info](#)

Use a load balancer to distribute network traffic across multiple servers. Enable service-to-service communications with VPC Lattice. Shift resources away from impaired Availability Zones with zonal shift. You can also customize health check replacements and monitoring.

Load balancing [Info](#)

Use the options below to attach your Auto Scaling group to an existing load balancer, or to a new load balancer that you define.

Select Load balancing options

☐ No load balancer
Traffic to your Auto Scaling group will not be fronted by a load balancer.

☒ Attach to an existing load balancer
Choose from your existing load balancers.

☐ Attach to a new load balancer
Quickly create a basic load balancer to attach to your Auto Scaling group.

Attach to an existing load balancer

Select the load balancers to attach

☒ Choose from your load balancer target groups
This option allows you to attach Application, Network, or Gateway Load Balancers.

☐ Choose from Classic Load Balancers

Existing load balancer target groups
Only instance target groups that belong to the same VPC as your Auto Scaling group are available for selection.

Select target groups

proj-app-tg | HTTP
Application Load Balancer: proj-alb

Final Take-Home Explanation (Corrected)

First, we created a VPC and imported it into Terraform to manage infrastructure as code without recreating resources.

Then, we created public and private subnets across multiple Availability Zones and associated appropriate route tables.

We attached an Internet Gateway to the VPC and configured the **public route table** with 0.0.0.0/0 → IGW, which makes the subnet public.

Next, we created a NAT Gateway inside a public subnet and attached an Elastic IP to it.

The **private route table** was configured with 0.0.0.0/0 → NAT Gateway,

allowing private EC2 instances outbound internet access without being publicly exposed.

After networking, we created an Application Load Balancer in public subnets and configured a Target Group.

The ALB listener forwards traffic to the Target Group, which routes requests to EC2 instances running in private subnets.

We installed NGINX on the private EC2 instance and validated the end-to-end traffic flow:

Internet → ALB → Target Group → Private EC2 → NGINX.

TROUBLESHOOTING (THIS PART IS EXCELLENT — MINOR REFINEMENT)

Your troubleshooting thinking is **correct**, just refine the order:

If application is not reachable from internet:

- 1** Check **ALB DNS** (does it resolve?)
- 2** Check **ALB Security Group**
 - Inbound: 80/443 from 0.0.0.0/0
 - 3** Check **Target Group health**
 - 4** Check **EC2 Security Group**
 - Allow inbound **only from ALB SG**, not from internet
 - 5** Check **NGINX is running**
 - 6** Check **route tables**
 - 7** Check **NACL only if SG is correct**

KEY PRODUCTION INSIGHT (VERY IMPORTANT)

In production, **backend EC2s live in private subnets** and are **never exposed directly to the internet**.

ALB acts as the **only public entry point**, and security groups enforce controlled access.

=====

CMD To verify Instance:

=====

EC2 INSTANCE VERIFICATION

=====

List all EC2 instances (any state)

aws ec2 describe-instances \

--query 'Reservations[].Instances[].InstanceId'

List only running EC2 instances

aws ec2 describe-instances \

--filters Name=instance-state-name,Values=running

=====

LOAD BALANCER (ALB)

=====

List all Application/Network Load Balancers

aws elbv2 describe-load-balancers \

--query 'LoadBalancers[].LoadBalancerName'

List all Target Groups

aws elbv2 describe-target-groups \

--query 'TargetGroups[].TargetGroupName'

=====

NAT GATEWAY

=====

List NAT Gateways (MOST IMPORTANT – PAID SERVICE)

aws ec2 describe-nat-gateways \

--query 'NatGateways[].NatGatewayId'

=====

ELASTIC IP (EIP)

=====

List all Elastic IPs

aws ec2 describe-addresses \

--query 'Addresses[].PublicIp'

=====

EBS VOLUMES

=====

List all EBS volumes

aws ec2 describe-volumes \

--query 'Volumes[].VolumeId'

List available (unattached) volumes

aws ec2 describe-volumes \

--filters Name=status,Values=available

=====

CLOUDWATCH

=====

List CloudWatch Log Groups

aws logs describe-log-groups \

--query 'logGroups[].logGroupName'

List CloudWatch Alarms

aws cloudwatch describe-alarms \

--query 'MetricAlarms[].AlarmName'

=====

TERRAFORM STATE

=====

Show all resources managed by Terraform

terraform state list

Destroy all Terraform-managed resources

terraform destroy

=====

FINAL ACCOUNT CLEAN CHECK

=====

EC2

aws ec2 describe-instances --query 'Reservations[].Instances[].InstanceId'

ALB

aws elbv2 describe-load-balancers --query
'LoadBalancers[].LoadBalancerName'

Target Groups

aws elbv2 describe-target-groups --query
'TargetGroups[].TargetGroupName'

NAT

aws ec2 describe-nat-gateways --query 'NatGateways[].NatGatewayId'

EIP

```
aws ec2 describe-addresses --query 'Addresses[].PublicIp'
```

EBS

```
aws ec2 describe-volumes --query 'Volumes[].VolumeId'
```

CloudWatch

```
aws logs describe-log-groups --query 'logGroups[].logGroupName'
```

```
aws cloudwatch describe-alarms --query 'MetricAlarms[].AlarmName'
```

=====

COMPLETE TERRAFORM CODE:

```
.
├── backend.tf
├── ec2.tf
├── main.tf
├── modules
│   ├── vpc
│   │   ├── alb.tf
│   │   ├── main.tf
│   │   ├── outputs.tf
│   │   └── variables.tf
├── provider.tf
├── terraform.tfstate.d
│   └── dev
│       ├── terraform.tfstate
│       ├── terraform.tfstate.1765789362.backup
│       ├── terraform.tfstate.1765789363.backup
│       └── terraform.tfstate.backup
├── terraform.tfvars
└── variables.tf
```

==

```
cat backend.tf
```

```
terraform {  
  required_version = ">= 1.5.0"
```

```
  
  required_providers {  
    aws = {  
      source = "hashicorp/aws"  
      version = "~> 5.0"  
    }  
  }  
}
```

```
  
cat ec2.tf  
resource "aws_instance" "app" {  
  ami           = var.app_ami_id  
  instance_type = var.app_instance_type  
  subnet_id     = var.app_subnet_id  
  vpc_security_group_ids = [var.app_sg_id]  
  key_name      = var.key_name
```

```
  
  tags = {  
    Name     = var.app_instance_name  
    Project  = "AWS-3Tier"  
    Environment = "dev"  
  }  
}
```

```
  
cat main.tf  
module "vpc" {  
  source = "../modules/vpc"
```

```
  
  vpc_id           = var.vpc_id  
  igw_id           = var.igw_id  
  nat_gateway_id   = var.nat_gateway_id  
  private_route_table_id = var.private_route_table_id
```

```
  
  alb_sg_id = var.alb_sg_id  
  app_sg_id = var.app_sg_id  
  db_sg_id  = var.db_sg_id
```

```
  
  public_subnet_ids = var.public_subnet_ids  
  public_subnets   = var.public_subnets  
  private_subnets  = var.private_subnets
```

```
  
  tags = var.tags  
}
```

```
  
cat provider.tf  
provider "aws" {  
  region = var.region  
}
```

```
  
cat terraform.tfvars  
app_ami_id = "ami-0b46816ffa1234887"
```

```
app_instance_type = "t3.micro"
app_subnet_id     = "subnet-03d9268f7580a2070"
app_sg_id         = "sg-002b5f602f4d511bc"
key_name          = "ThreeTierApp"
app_instance_name = "proj-app-ec2"
```

```
region = "eu-north-1"
```

```
alb_sg_id = "sg-0ce7e7640c387f725"
```

```
public_subnet_ids = [
    "subnet-0e123e973c24fdecf",
    "subnet-04a9f9d40c1e98614"
]
```

```
# =====
# SECURITY GROUP IDS
# (existing – no creation)
# =====
```

```
# ALB / Public-facing SG
#public_sg_id = "sg-0ce7e7640c387f725"
```

```
# Application tier SG
#app_sg_id = "sg-002b5f602f4d511bc"
```

```
# Database tier SG
db_sg_id = "sg-09b4fa7e45512a99a"
```

```
nat_gateway_id     = "nat-1c803e8cbf90c9ada"
private_route_table_id = "rtb-01950229dbfa4a057"
```

```
vpc_id = "vpc-0d06a3a644a12185a"
igw_id = "igw-01952ba2b33377693"
```

```
#####
# PUBLIC SUBNETS (ONLY)
#####
```

```
public_subnets = {
    "proj-public-subnet-1" = {
        cidr = "10.0.1.0/24"
        az   = "eu-north-1a"
    }
}
```

```
"proj-public-subnet-2" = {
    cidr = "10.0.2.0/24"
    az   = "eu-north-1b"
}
}
```

```
#####
```

```
# PRIVATE SUBNETS (ONLY)
#####

private_subnets = {
  "proj-private-subnet-1" = {
    cidr = "10.0.11.0/24"
    az   = "eu-north-1a"
  }

  "proj-private-subnet-2" = {
    cidr = "10.0.12.0/24"
    az   = "eu-north-1a" # MUST match AWS
  }
}
```

```
#####
# TAGS (OPTIONAL – MUST MATCH AWS)
#####
tags = {
  Project   = "AWS-3Tier"
  Environment = "dev"
}
```

```
cat variables.tf
variable "app_ami_id" {
  type = string
}
```

```
variable "app_instance_type" {
  type = string
}
```

```
variable "app_subnet_id" {
  type = string
}
```

```
variable "app_sg_id" {
  type = string
}
```

```
variable "key_name" {
  type = string
}
```

```
variable "app_instance_name" {
  type = string
}
```

```
#####
# GLOBAL
#####
variable "region" {
  description = "AWS region"
  type        = string
}
```

```
variable "tags" {
  description = "Common tags for all resources"
  type       = map(string)
}
```

```
#####
# NETWORK (EXISTING)
#####
variable "vpc_id" {
  description = "Existing VPC ID"
  type       = string
}
```

```
variable "igw_id" {
  description = "Existing Internet Gateway ID"
  type       = string
}
```

```
variable "nat_gateway_id" {
  description = "Existing NAT Gateway ID"
  type       = string
}
```

```
variable "private_route_table_id" {
  description = "Existing private route table ID"
  type       = string
}
```

```
#####
# SUBNET DEFINITIONS
#####
variable "public_subnets" {
  description = "Public subnet definitions"
  type       = map(object({
    cidr = string
    az   = string
  }))
}
```

```
variable "private_subnets" {
  description = "Private subnet definitions"
  type       = map(object({
    cidr = string
    az   = string
  }))
}
```

```
variable "public_subnet_ids" {
  description = "Public subnet IDs (used by ALB)"
  type       = list(string)
}
```

```
#####
# SECURITY GROUP IDS (EXISTING)
#####
variable "alb_sg_id" {
  description = "ALB security group ID"
  type       = string
}
```

```
}

#variable "app_sg_id" {
#  description = "Application security group ID"
#  type      = string
#}
```

```
variable "db_sg_id" {
  description = "Database security group ID"
  type      = string
}
```

```
cat alb.tf
#####
# APPLICATION LOAD BALANCER
#####
```

```
resource "aws_lb" "alb" {
  name          = "proj-alb"
  load_balancer_type = "application"
  subnets      = var.public_subnet_ids
  security_groups = [var.alb_sg_id]
```

```
#  lifecycle {
#    prevent_destroy = true
#  }
}
```

```
#####
# TARGET GROUP (NO TARGETS)
#####
```

```
resource "aws_lb_target_group" "app" {
  name     = "proj-app-tg"
  port     = 80
  protocol = "HTTP"
  vpc_id   = var.vpc_id
  target_type = "instance"
```

```
lambda_multi_value_headers_enabled = false
proxy_protocol_v2                   = false
```

```
health_check {
  enabled      = true
  interval     = 30
  path         = "/"
  port         = "traffic-port"
  protocol     = "HTTP"
  healthy_threshold = 5
  unhealthy_threshold = 2
  timeout      = 5
  matcher      = "200"
}
```

```

tags = {
  Name    = "proj-app-tg"
  Project = "AWS-3Tier"
  Environment = "dev"
}
}

```

```

#####
# HTTP LISTENER (ONLY HTTP)
#####

```

```

resource "aws_lb_listener" "http" {
  load_balancer_arn = "arn:aws:elasticloadbalancing:eu-north-1:211811255273:loadbalancer/app/proj-alb/7ad611ec77590724"
  port              = 80
  protocol          = "HTTP"
}

```

```

default_action {
  type = "forward"
}

```

```

forward {
  target_group {
    arn = "arn:aws:elasticloadbalancing:eu-north-1:211811255273:targetgroup/proj-app-tg/26ddc4e3c02e1e5d"
    weight = 1
  }
}

```

```

stickiness {
  enabled = false
  duration = 3600
}
}
}

```

```

# lifecycle {
#   prevent_destroy = true
#
#   ignore_changes = [
#     default_action, # ← THIS is the key
#     tags
#   ]
# }
}

```

```

arpitchouksey@Arpits-MacBook-Air vpc %
arpitchouksey@Arpits-MacBook-Air vpc % cat main.tf

```

```

#####
# PUBLIC / ALB SECURITY GROUP
#####
resource "aws_security_group" "public" {
  vpc_id = var.vpc_id
  name    = "proj-alb-sg"
  description = "ALB security group"
}

```

```

# lifecycle {
#   ignore_changes = [
#     ingress,
#     egress,

```



```

#   description
# ]
# }

tags = merge(var.tags, {
  Name = "proj-alb-sg"
})
}

#####
# APP SECURITY GROUP
#####
resource "aws_security_group" "app" {
  vpc_id   = var.vpc_id
  name     = "proj-app-sg"
  description = "Application EC2 security group"

# lifecycle {
#   ignore_changes = [
#     ingress,
#     egress,
#     description
#   ]
# }

tags = merge(var.tags, {
  Name = "proj-app-sg"
})
}

#####
# DB SECURITY GROUP
#####
resource "aws_security_group" "db" {
  vpc_id   = var.vpc_id
  name     = "proj-db-sg"
  description = "Allow ec2 to edit "

# lifecycle {
#   ignore_changes = [
#     ingress,
#     egress,
#     description
#   ]
# }

tags = merge(var.tags, {
  Name = "proj-db-sg"
})
}

#####
# EXISTING VPC (DATA SOURCE)
#####
data "aws_vpc" "this" {
  id = var.vpc_id

```

```

}

#####
# EXISTING IGW (DATA SOURCE)
#####

data "aws_internet_gateway" "this" {
  filter {
    name = "internet-gateway-id"
    values = [var.igw_id]
  }
}

#####
# PRIVATE INTERNET ROUTE (NAT)
#####
resource "aws_route" "private_nat" {
  route_table_id = var.private_route_table_id
  destination_cidr_block = "0.0.0.0/0"
  nat_gateway_id = var.nat_gateway_id
}

#####
# PUBLIC SUBNETS
#####
resource "aws_subnet" "public" {
  for_each = var.public_subnets

  vpc_id = data.aws_vpc.this.id
  cidr_block = each.value.cidr
  availability_zone = each.value.az
  map_public_ip_on_launch = true

  tags = merge(
    var.tags,
    { Name = each.key }
  )
}

#####
# PRIVATE SUBNETS
#####
resource "aws_subnet" "private" {
  for_each = var.private_subnets

  vpc_id = data.aws_vpc.this.id
  cidr_block = each.value.cidr
  availability_zone = each.value.az

  tags = merge(
    var.tags,
    { Name = each.key }
  )
}

```

```
#####
# PUBLIC ROUTE TABLE
#####
resource "aws_route_table" "public" {
  vpc_id = data.aws_vpc.this.id

  tags = merge(
    var.tags,
    { Name = "proj-public-rt" }
  )
}

#####
# INTERNET ROUTE (NO CHANGE)
#####
resource "aws_route" "public_internet" {
  route_table_id      = aws_route_table.public.id
  destination_cidr_block = "0.0.0.0/0"
  gateway_id          = data.aws_internet_gateway.this.id
}

#####
# ROUTE TABLE ASSOCIATIONS
#####
resource "aws_route_table_association" "public" {
  for_each = aws_subnet.public

  subnet_id      = each.value.id
  route_table_id = aws_route_table.public.id
}

arpitchouksey@Arpits-MacBook-Air vpc % cat outputs.tf
output "public_subnet_ids" {
  value = values(aws_subnet.public)[*].id
}

output "private_subnet_ids" {
  value = values(aws_subnet.private)[*].id
}

arpitchouksey@Arpits-MacBook-Air vpc % cat variables.tf
variable "alb_sg_id" {
  type      = string
  description = "ALB security group ID"
}

variable "public_subnet_ids" {
  type      = list(string)
  description = "Public subnet IDs for ALB"
}

#variable "public_sg_id" {
#  description = "Existing Public Security Group ID"
#  type      = string
#}
```

```
variable "app_sg_id" {
  description = "Existing App Security Group ID"
  type       = string
}
```

```
variable "db_sg_id" {
  description = "Existing DB Security Group ID"
  type       = string
}
```

```
variable "nat_gateway_id" {
  description = "Existing NAT Gateway ID"
  type       = string
}
```

```
variable "private_route_table_id" {
  description = "Existing private route table ID"
  type       = string
}
```

```
variable "vpc_id" {
  type = string
}
```

```
variable "igw_id" {
  description = "Internet Gateway ID for public route"
  type       = string
}
```

```
variable "public_subnets" {
  type = map(object({
    cidr = string
    az   = string
  }))
}
```

```
variable "private_subnets" {
  type = map(object({
    cidr = string
    az   = string
  }))
}
```

```
variable "tags" {
  type = map(string)
}
```

